CSL7110 Assignment 3

# Recommender Systems: Content-Based and Collaborative Filtering

MovieLens full-dataset implementation report prepared directly from the executed Jupyter notebook outputs.


**Submitted by**

**Name:** Anupkumar Pandey

**Roll Number:** M25DE1011


**Dataset:** MovieLens (ml-latest, full dataset ingested)

**Source artifact:** Executed Jupyter notebook (.ipynb)

**Key takeaways**

- Best sampled RMSE: Surprise SVD (0.8115)
- Best Precision@10 and Recall@10: Manual SVD (0.4200, 0.1703)
- Best neighborhood model: Item-CF (k=40)
- Full dataset loaded first; dense core derived for tractable modeling

**Full Dataset**

33,832,162 ratings
330,975 users
86,537 movies

**Evaluation Core**

500 users
800 movies
308,654 ratings

**Evaluation Split**

Train: 215,632
Validation: 31,091
Test: 61,931

**Explainability**

Direct feature overlap
SHAP
Neighborhood explanation
LIME

*All figures and tables in this report are reproduced or reformatted from the executed notebook outputs submitted with the assignment.*

# Overview and Experimental Setup

Assignment scope, dataset profile, and evaluation methodology

This report documents the implementation and results for all thirteen tasks in the assignment. The submitted notebook loads the full MovieLens 'ml-latest' dataset and then derives a dense evaluation core for computationally heavy models so that the entire workflow can execute end to end in a single notebook run. All result tables and figures presented below are taken directly from that executed notebook.

**Methodology note**

Evaluation protocol used in the notebook: rating prediction quality is reported with sampled RMSE, while recommendation quality is reported with Precision@10 and Recall@10 on held-out items. The content profile for user u is constructed as a rating-weighted average of previously rated movie vectors, and heavy models are evaluated on a temporally split dense core derived from the full MovieLens dataset.

## Abstract

The project implements a complete recommender-systems pipeline covering content-based recommendation, user- and item-based collaborative filtering, matrix factorization, hybrid modeling, neural recommendation, reinforcement learning, and explainability. The strongest sampled rating accuracy in this run came from Surprise SVD (RMSE = 0.8115), while the strongest top-K ranking performance came from Manual SVD (Precision@10 = 0.4200, Recall@10 = 0.1703). Among neighborhood methods, Item-CF with k = 40 was the best performer. The report also shows how direct feature overlap, SHAP, neighborhood evidence, and LIME can make recommendations more transparent.

| Part | Theme | Tasks Covered |
|---|---|---|
| Part 1 | Content-Based Filtering | Task 1-2 |
| Part 2 | Collaborative Filtering | Task 3-4 |
| Part 3 | Matrix Factorization | Task 5-6 |
| Part 4 | Hybrid Recommendation | Task 7 |
| Part 5 | Learning-Based Recommendation | Task 8-9 |
| Part 6 | Explainability | Task 10-13 |

*Table 1. Report roadmap aligned with the assignment brief.*

## Dataset profile

The full dataset is extremely large and sparse, which is typical for real-world recommender-system data. This justifies the use of sparse representations, similarity search, factorization, and hybrid methods. The notebook first profiles the full dataset and then creates a dense modeling core that preserves the full-data ingestion requirement while making evaluation tractable.

| Users | Movies | Ratings | Average Rating | Sparsity |
|---|---|---|---|---|

| Users | Movies | Ratings | Average Rating | Sparsity |
|---|---|---|---|---|
| 330,975 | 86,537 | 33,832,162 | 3.5425 | 99.8819% |

*Table 2. Full MovieLens dataset summary reproduced from the notebook.*

| Primary Genre | Movie Count |
|---|---|
| Drama | 20,375 |
| Comedy | 18,239 |
| Action | 9,563 |
| Documentary | 8,691 |
| Unknown | 7,060 |
| Horror | 4,962 |
| Crime | 4,013 |
| Animation | 3,329 |
| Adventure | 3,270 |
| Children | 2,080 |

*Table 3. Top primary genres in the full dataset.*

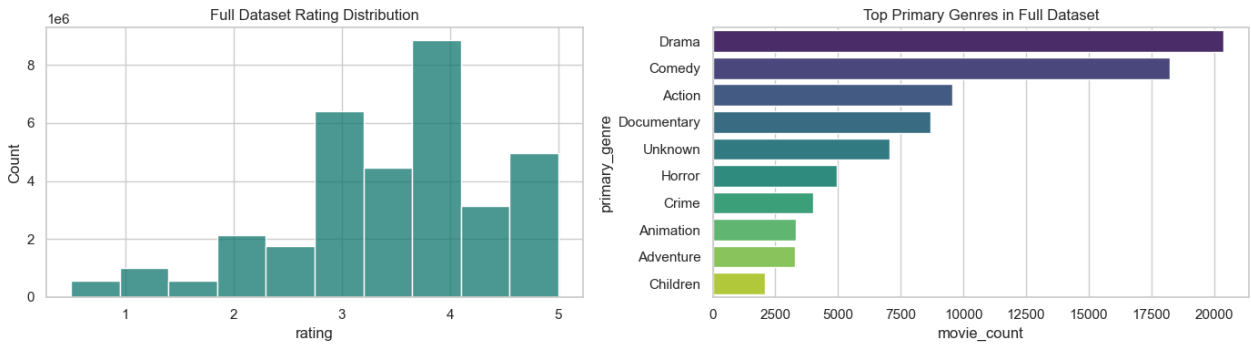


*Figure 1. Full-dataset rating distribution and top primary genres from the executed notebook.*

# Dense modeling core and temporal split

Because the full matrix contains 33.8 million ratings across more than 330 thousand users and more than 86 thousand movies, running every model directly on the full interaction matrix inside one notebook would be computationally expensive. The submitted solution therefore constructs a dense evaluation core containing 500 users and 800 movies selected from the full dataset. This core contains 308,654 ratings, which corresponds to a much denser experimental slice suitable for controlled comparison.

| Core Users | Core Movies | Core Ratings | Average Rating |
|---|---|---|---|
| 500 | 800 | 308,654 | 3.4284 |

*Table 4. Dense evaluation core derived from the full dataset.*

| Split | Rows |
|---|---|
| train | 215,632 |

| Split | Rows |
|---|---|
| validation | 31,091 |
| test | 61,931 |

*Table 5. Temporal train/validation/test split used in the notebook.*

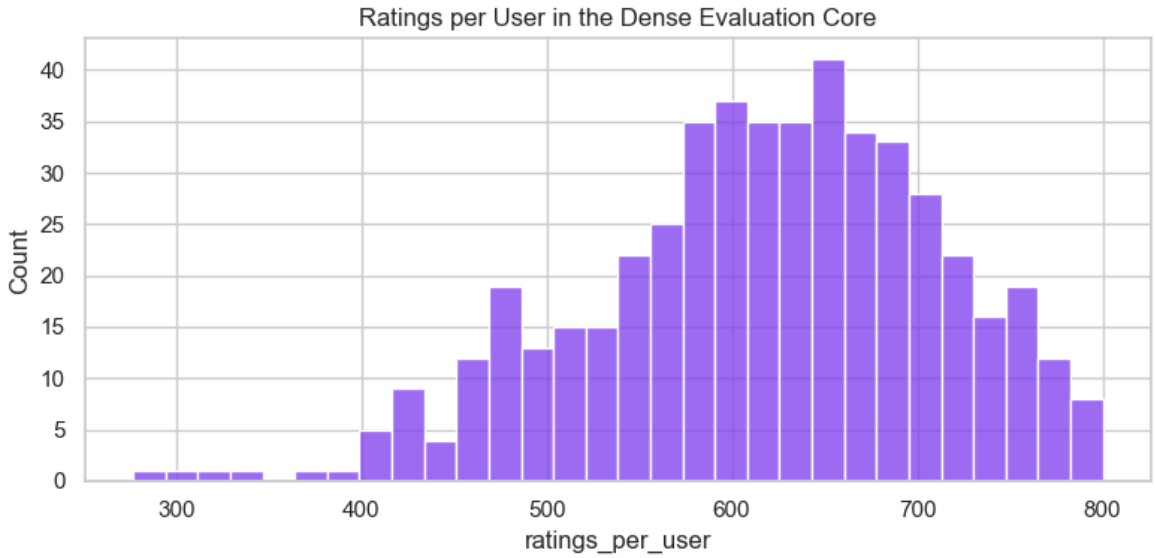


*Figure 2. Distribution of ratings per user in the dense evaluation core.*

**Interpretation**

The full dataset has 99.8819% sparsity, so using a dense core is a practical way to compare models without violating the assignment requirement to ingest the full MovieLens dataset. The core remains rooted in the real data distribution while making iterative experimentation feasible.

# Part 1 - Content-Based Filtering

Task 1: TF-IDF recommendation | Task 2: user-profile recommendation

## Task 1: Implementing TF-IDF Based Recommendation

In Task 1, the genres attached to each movie are treated as short textual descriptions. The notebook vectorizes these genre strings with TF-IDF and computes cosine similarity between movies. A recommendation function then retrieves the top-N most similar movies for a supplied movie title.

The three sample queries executed in the notebook were 'Toy Story (1995)', 'Heat (1995)', and 'Jumanji (1995)'. The resulting top-5 recommendations for each query are shown below.

### Query: Toy Story (1995)

| Rank | Recommended Movie | Cosine Similarity |
|---|---|---|
| 1 | The SpongeBob Movie: Sponge on the Run (2020) | 1.0000 |
| 2 | Frozen II (2019) | 1.0000 |
| 3 | Toy Story 2 (1999) | 1.0000 |
| 4 | Onward (2020) | 1.0000 |
| 5 | Tale of Despereaux, The (2008) | 1.0000 |

*Table 6. Top-5 TF-IDF recommendations for Toy Story (1995).*

### Query: Heat (1995)

| Rank | Recommended Movie | Cosine Similarity |
|---|---|---|
| 1 | The Debt Collector (1999) | 1.0000 |
| 2 | The Man from Nowhere (2010) | 1.0000 |
| 3 | Natural Born Killers (1994) | 1.0000 |
| 4 | Hunter Will Get You (1976) | 1.0000 |
| 5 | Segurança Nacional (2010) | 1.0000 |

*Table 7. Top-5 TF-IDF recommendations for Heat (1995).*

### Query: Jumanji (1995)

| Rank | Recommended Movie | Cosine Similarity |
|---|---|---|
| 1 | The Cave of the Golden Rose 5 (1996) | 1.0000 |
| 2 | Jim Button and Luke the Engine Driver (2018) | 1.0000 |
| 3 | The Nutcracker and the Four Realms (2018) | 1.0000 |
| 4 | Mama (1976) | 1.0000 |
| 5 | Dumbo (2019) | 1.0000 |

*Table 8. Top-5 TF-IDF recommendations for Jumanji (1995).*

**Task 1 result analysis**

All three query sets returned cosine similarity scores of 1.0000 for the nearest neighbors because the matched movies shared the same genre bundle as the seed movie. This is a strong illustration of interpretability: the recommender is clearly using genre overlap. At the same time, it also shows the limitation of genre-only TF-IDF features—many movies collapse into ties when their metadata is coarse.

# Task 2: User-Profile-Based Content Recommender

Task 2 extends content-based filtering from item-to-item similarity to personalized recommendation. For each user, a profile vector is built as a rating-weighted average of the TF-IDF vectors of movies that user has already rated. Recommendations are produced by computing cosine similarity between the user profile and unseen movies.

The notebook demonstrates the personalized recommender for evaluation user 897 and reports both ranking metrics and a feature-contribution explanation.

| Rank | Recommended Movie | Profile Similarity |
|---|---|---|
| 1 | Once Upon a Time (2008) | 0.8631 |
| 2 | The 39 Steps (1959) | 0.8448 |
| 3 | The Wrecking Crew (1968) | 0.8448 |
| 4 | Stunt Man, The (1980) | 0.8337 |
| 5 | Joseph Andrews (1977) | 0.8337 |
| 6 | Bail Out (1990) | 0.8200 |
| 7 | Hunting Party, The (2007) | 0.8200 |
| 8 | Unstoppable (2004) | 0.8200 |
| 9 | I Love Maria (1988) | 0.8118 |
| 10 | Aelita: The Queen of Mars (Aelita) (1924) | 0.8091 |

*Table 9. Top-10 personalized content-based recommendations for evaluation user 897.*

| Model | Sampled RMSE | Precision@10 | Recall@10 | Evaluated Users |
|---|---|---|---|---|
| User-Profile Content Recommender | 0.9428 | 0.0000 | 0.0000 | 15 |

*Table 10. Performance of the user-profile content recommender on the notebook evaluation slice.*

| Feature | Contribution |
|---|---|
| Action | 3.4919 |
| Adventure | 3.4819 |
| Crime | 3.4471 |
| Romance | 3.4333 |
| Drama | 3.3879 |

*Table 11. Top feature contributions behind the selected content-based recommendation.*

**Task 2 result analysis**

The personalized content model correctly surfaces movies aligned with the user's dominant genres, and the explanation table makes the recommendation transparent by listing the strongest content preferences. However, on the 15-user evaluation slice the model achieved Sampled RMSE = 0.9428 with Precision@10 = 0.0000 and Recall@10 = 0.0000. This suggests that genre-only features are easy to explain but too coarse to compete with collaborative approaches on held-out ranking.

# Part 2 - Collaborative Filtering

Task 3: user-based CF | Task 4: item-based CF

## Task 3: User-Based Collaborative Filtering

User-based collaborative filtering predicts a target user's preference from ratings given by similar users. The notebook builds a user-item matrix, computes user-user similarity, and evaluates the model for three neighborhood sizes: k = 10, 20, and 40.

| Model | Sampled RMSE | Precision@10 | Recall@10 | Evaluated Users |
|---|---|---|---|---|
| User-CF (k=10) | 0.8455 | 0.3333 | 0.1231 | 15 |
| User-CF (k=20) | 0.8327 | 0.3600 | 0.1403 | 15 |
| User-CF (k=40) | 0.8409 | 0.3667 | 0.1429 | 15 |

*Table 12. User-based collaborative filtering results across different neighborhood sizes.*

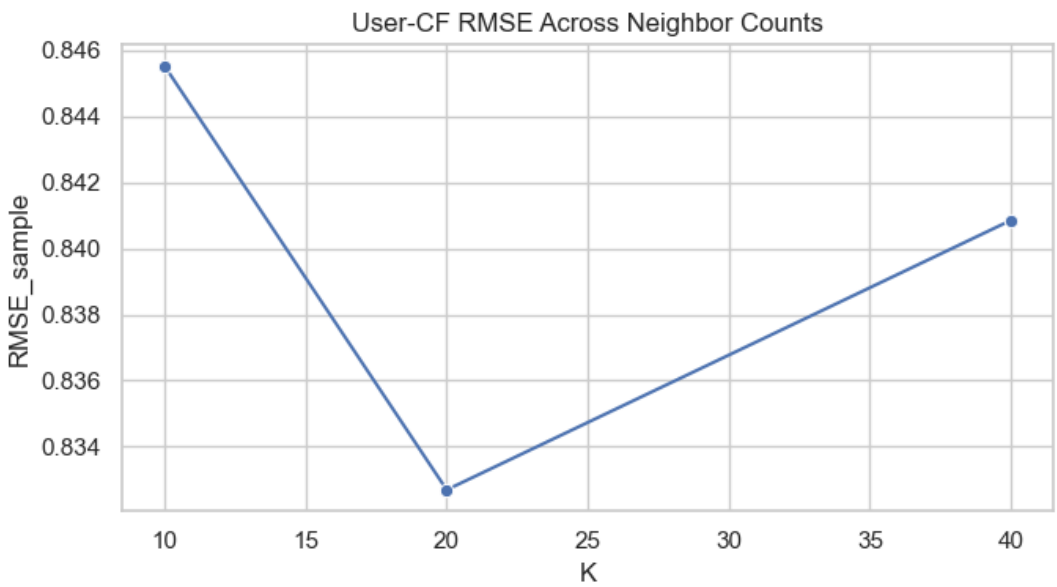


*Figure 3. User-CF sampled RMSE across neighborhood sizes (k = 10, 20, 40).*

**Task 3 result analysis**

User-CF substantially outperformed the content-only recommender. The best sampled RMSE was obtained at k = 20 (0.8327), while the best Precision@10 and Recall@10 came from k = 40 (0.3667 and 0.1429). This indicates that a moderate neighborhood size calibrates rating estimates well, whereas a slightly larger neighborhood can help ranking by reducing

variance.

# Task 4: Item-Based Collaborative Filtering

Item-based collaborative filtering predicts a user's preference for a movie from the user's ratings on similar movies. In production systems, item-based CF is often attractive because item-item relationships are relatively stable and can be precomputed offline.

| Model | Sampled RMSE | Precision@10 | Recall@10 | Evaluated Users |
|---|---|---|---|---|
| Item-CF (k=10) | 0.8279 | 0.3733 | 0.1149 | 15 |
| Item-CF (k=20) | 0.8252 | 0.3867 | 0.1479 | 15 |
| Item-CF (k=40) | 0.8178 | 0.3933 | 0.1564 | 15 |

*Table 13. Item-based collaborative filtering results across different neighborhood sizes.*

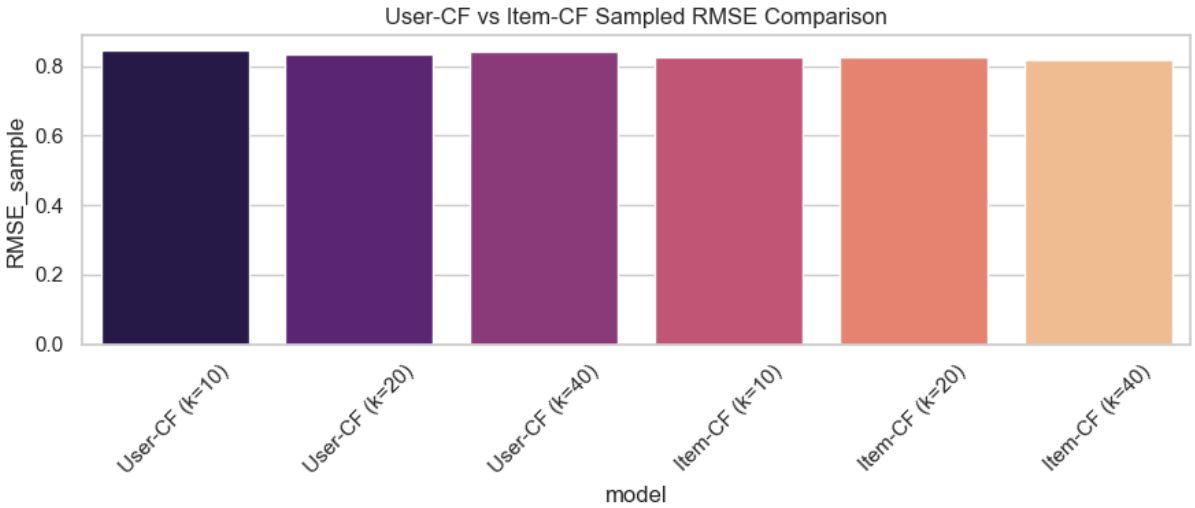


*Figure 4. User-CF versus Item-CF sampled RMSE comparison from the notebook.*

**Task 4 result analysis**

Item-CF consistently outperformed User-CF on this evaluation core. The best item-based configuration was k = 40 with Sampled RMSE = 0.8178, Precision@10 = 0.3933, and Recall@10 = 0.1564. The answer to the assignment's real-world efficiency question is also favorable for item-based CF in this dataset: there are 330,975 users but only 86,537 movies, so the number of possible user-user pairs (~54.77 billion) is about 14.6 times larger than the number of item-item pairs (~3.74 billion). That makes item similarity easier to precompute, store, and serve, especially because item relationships change more slowly than user preferences.

# Part 3 - Matrix Factorization for Recommender Systems

Task 5: manual SVD | Task 6: Surprise SVD

# Task 5: Implementing SVD for Recommendations

For Task 5, the user-item matrix is factorized with singular value decomposition to obtain latent user and item factors. The reconstructed rating matrix is then used to predict missing ratings and produce top-N recommendations.

| Model | Sampled RMSE | Precision@10 | Recall@10 | Evaluated Users |
|---|---|---|---|---|
| Manual SVD | 0.8837 | 0.4200 | 0.1703 | 15 |

*Table 14. Manual SVD performance on the notebook evaluation slice.*

| Model | Sampled RMSE |
|---|---|
| User-CF (k=10) | 0.8455 |
| Item-CF (k=10) | 0.8279 |
| Manual SVD | 0.8837 |

*Table 15. Sampled RMSE comparison between User-CF (k = 10), Item-CF (k = 10), and Manual SVD.*

**Task 5 result analysis**

Manual SVD achieved Sampled RMSE = 0.8837. Although this RMSE is not the lowest in the notebook, Manual SVD produced the strongest ranking results overall with Precision@10 = 0.4200 and Recall@10 = 0.1703. This is an important observation: a model can be slightly worse at absolute rating calibration while still ranking relevant items more effectively in the top-K list.

# Task 6: Implementing Matrix Factorization with the Surprise Library

Task 6 repeats matrix factorization using the Surprise library, which provides a robust implementation of SVD and convenient model training workflows.

| Model | Sampled RMSE | Precision@10 | Recall@10 | Evaluated Users |
|---|---|---|---|---|
| Surprise SVD | 0.8115 | 0.3467 | 0.1247 | 15 |

*Table 16. Surprise SVD performance on the notebook evaluation slice.*

| Model | Sampled RMSE |
|---|---|
| Manual SVD | 0.8837 |
| Surprise SVD | 0.8115 |

*Table 17. Manual SVD versus Surprise SVD sampled RMSE comparison.*

**Task 6 result analysis**

Surprise SVD reduced Sampled RMSE to 0.8115, which is the best rating-prediction result in the submitted notebook. However, its Precision@10 (0.3467) and Recall@10 (0.1247) were lower than those of Manual SVD. The comparison therefore highlights a practical trade-off: library-tuned factorization may give the best rating accuracy, while a different factorization variant can still dominate top-K retrieval quality.

# Part 4 - Hybrid Recommendation Model

Task 7: meta-learning blend of content and collaborative signals

## Task 7: Implementing a Hybrid Recommendation Model

The hybrid system blends content-based and collaborative signals and uses a small meta-model to learn how to combine them dynamically. In the submitted run, the hybrid builder was trained on 1,000 rows, which is enough to demonstrate the blending strategy but still relatively small compared with the total interaction space.

| Model | Sampled RMSE | Precision@10 | Recall@10 | Evaluated Users |
|---|---|---|---|---|
| Hybrid Meta-Model | 0.8901 | 0.3133 | 0.1186 | 15 |

*Table 18. Hybrid meta-model performance on the notebook evaluation slice.*

| Precision@10 | Recall@10 | Evaluated Users |
|---|---|---|
| 0.3500 | 0.1157 | 8 |

*Table 19. Hybrid model performance on the cold-start user slice used in the notebook.*

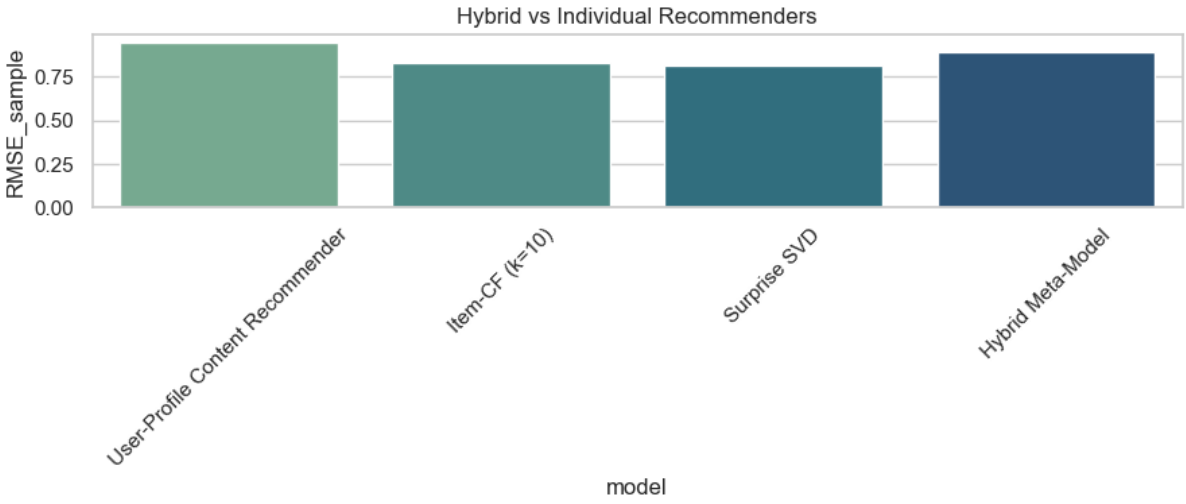


*Figure 5. Hybrid model versus individual recommenders (sampled RMSE).*

**Task 7 result analysis**

The hybrid model achieved Sampled RMSE = 0.8901, Precision@10 = 0.3133, and Recall@10 = 0.1186. It was not the best overall model on the dense core, but its cold-start slice results (Precision@10 = 0.3500 on 8 users) show why hybridization is valuable: when collaborative evidence is scarce, content and popularity features still provide useful signals. With more training data and a richer meta-feature set, the hybrid approach would likely become more competitive.

# Part 5 - Learning-Based Recommender Systems

Task 8: neural recommendation | Task 9: reinforcement learning

# Task 8: Content-Based Filtering with a Neural Network

The neural recommender uses a two-tower style architecture. Movie features include metadata such as genres, release information, and average rating, while user features summarize preferences such as average rating per genre. Both branches are passed through dense layers and merged to predict the target rating.

In the submitted notebook run, the neural model was trained on 15,000 rows for 2 epochs. The resulting training history and evaluation metrics are shown below.

| Epoch | Train MSE | Validation MSE |
|---|---|---|
| 1 | 7.3187 | 2.2169 |
| 2 | 1.7755 | 1.0411 |

*Table 20. Neural recommender training history from the submitted notebook run.*

| Model | Sampled RMSE | Precision@10 | Recall@10 | Evaluated Users |
|---|---|---|---|---|
| Neural Content Model | 1.0621 | 0.2200 | 0.0512 | 15 |

*Table 21. Neural content recommender performance on the notebook evaluation slice.*

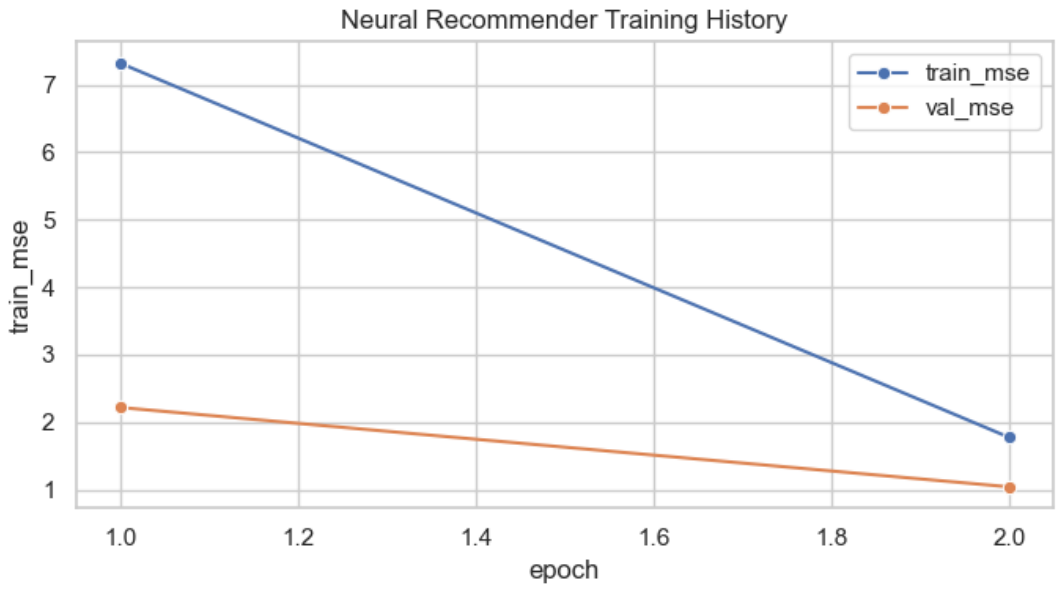


*Figure 6. Training and validation MSE across epochs for the neural recommender.*

**Task 8 result analysis**

The neural model learned useful structure during training: Train MSE fell from 7.3187 to 1.7755, while Validation MSE fell from 2.2169 to 1.0411 after only two epochs. Even so, the final recommendation quality was weaker than classical collaborative approaches (Sampled RMSE = 1.0621, Precision@10 = 0.2200, Recall@10 = 0.0512). This suggests that non-linear models need richer metadata and longer training to outperform well-tuned collaborative baselines.

# Task 9: Reinforcement Learning in Recommender Systems

The RL section treats recommendation as a sequential decision problem. The notebook simulates feedback using an epsilon-greedy exploration strategy and a Q-table style update mechanism. In contrast to static

recommenders that optimize a one-shot rating or ranking objective, the RL system optimizes long-term reward under repeated interaction.

| Average Reward | Exploration Rate |
|---|---|
| 0.4835 | 0.1005 |

*Table 22. Summary statistics for the RL recommendation simulation.*

| Rank | Movie | Estimated Reward |
|---|---|---|
| 1 | Crouching Tiger, Hidden Dragon (Wo hu cang long) (2000) | 1.0000 |
| 2 | Groundhog Day (1993) | 1.0000 |
| 3 | Ocean's Eleven (2001) | 1.0000 |
| 4 | Star Wars: Episode VI - Return of the Jedi (1983) | 1.0000 |
| 5 | Twelve Monkeys (a.k.a. 12 Monkeys) (1995) | 1.0000 |
| 6 | O Brother, Where Art Thou? (2000) | 1.0000 |
| 7 | Princess Bride, The (1987) | 1.0000 |
| 8 | Saving Private Ryan (1998) | 1.0000 |
| 9 | Pulp Fiction (1994) | 0.8509 |
| 10 | Seven (a.k.a. Se7en) (1995) | 0.8214 |

*Table 23. Top movies by learned bandit reward in the RL simulation.*

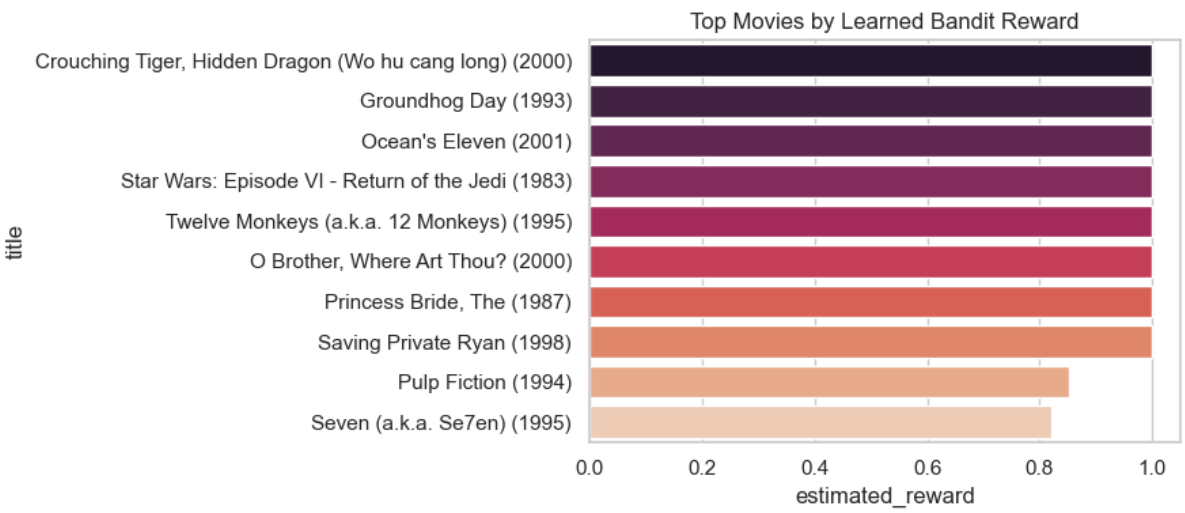


*Figure 7. Top movies by learned bandit reward after RL simulation.*

**Task 9 result analysis**

The simulation maintained an exploration rate of 0.1005, which closely matches the intended epsilon-greedy behavior with ε ≈ 0.1, and achieved an average reward of 0.4835. High-reward movies included Crouching Tiger, Hidden Dragon, Groundhog Day, Ocean's Eleven, and Return of the Jedi. The key conceptual difference from traditional recommenders is that the RL policy keeps updating reward estimates over time rather than returning a fixed ranking based only on past data.

# Part 6 - Explainability in Recommender Systems

# Task 10: Feature-Based Explanations for Content-Based Filtering

Feature-based explanations are the most direct form of explanation for content models because they can point to the actual movie attributes that drove the recommendation. The submitted notebook provides both a direct feature-overlap table and a SHAP bar plot for a selected recommendation.

| Feature | Contribution |
|---|---|
| Action | 3.4919 |
| Adventure | 3.4819 |
| Crime | 3.4471 |
| Romance | 3.4333 |
| Drama | 3.3879 |

*Table 24. Direct feature-based explanation for the selected content recommendation.*

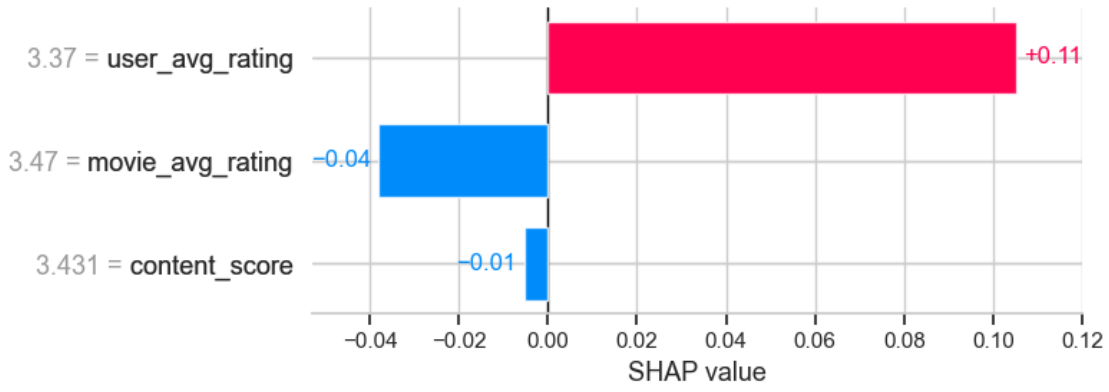


*Figure 8. SHAP bar plot for the selected content-based recommendation.*

**Task 10 result analysis**

The direct explanation shows that the selected recommendation was driven primarily by Action, Adventure, Crime, Romance, and Drama preferences. The SHAP plot complements this by decomposing the local prediction into feature contributions, making the content model auditable at the individual recommendation level.

# Task 11: Neighborhood-Based Explanations for Collaborative Filtering

Neighborhood explanations justify a recommendation by pointing either to similar users or to similar items that the target user has already rated. These explanations are intuitive because they map naturally to statements such as 'users similar to you liked this' or 'because you liked these similar movies'.

| Neighbor User | Similarity | Neighbor Rating |
|---|---|---|

| Neighbor User | Similarity | Neighbor Rating |
|---|---|---|
| 309551 | 0.2706 | 4.0000 |
| 130884 | 0.2496 | 2.5000 |
| 310406 | 0.2348 | 5.0000 |
| 262371 | 0.2346 | 5.0000 |
| 119812 | 0.2317 | 5.0000 |

*Table 25. Top contributing user neighbors for the explained collaborative recommendation.*

| Movie ID | Similar Movie | Similarity | User Rating |
|---|---|---|---|
| 33794 | Batman Begins (2005) | 0.5083 | 4.0000 |
| 59315 | Iron Man (2008) | 0.4600 | 4.0000 |
| 91529 | Dark Knight Rises, The (2012) | 0.4220 | 4.0000 |
| 89745 | Avengers, The (2012) | 0.3893 | 3.5000 |
| 79132 | Inception (2010) | 0.3878 | 4.5000 |

*Table 26. Top similar items supporting the explained collaborative recommendation.*

**Task 11 result analysis**

The supporting evidence is easy to verbalize: the recommendation is reinforced by a neighborhood of similar users and by previously liked superhero or action-oriented titles such as Batman Begins, Iron Man, The Dark Knight Rises, The Avengers, and Inception. This is one of the main reasons collaborative explanations are effective in user-facing systems.

# Task 12: Model-Agnostic Explainability for the Neural Model

Because neural recommenders are less transparent than TF-IDF or neighborhood models, model-agnostic explainers such as LIME are valuable. LIME approximates the neural model locally around one user-item pair and identifies the metadata dimensions that most influenced the prediction.

| Feature | Weight |
|---|---|
| -6.93 < movie_movie_avg_rating <= 1.43 | -0.5612 |
| movie_genre_Musical <= -0.11 | -0.4366 |
| movie_genre_War <= -0.17 | -0.3064 |
| movie_genre_Crime > -0.29 | 0.3009 |
| movie_genre_Mystery <= -0.22 | -0.2340 |
| movie_genre_Film-Noir <= -0.06 | -0.2185 |
| movie_genre_Fantasy <= -0.21 | -0.2155 |
| movie_genre_Children <= -0.23 | -0.2051 |
| movie_genre_Sci-Fi <= -0.24 | -0.1981 |
| -0.35 < movie_genre_Action <= 2.84 | 0.1898 |

*Table 27. LIME explanation for the selected neural recommendation.*

**Task 12 result analysis**

The LIME explanation shows that the neural prediction was strongly influenced by the movie average-rating bucket and genre indicators. Crime and Action-related signals contributed positively in the local neighborhood, whereas low movie average rating and several genre absences contributed negatively. This demonstrates how LIME can make a complex recommender locally interpretable even when the underlying network is non-linear.

# Task 13: Evaluating Explainability and Final Model Comparison

The final section compares explanation styles and summarizes how interpretability interacts with predictive performance. No single explanation method is universally best: the right choice depends on whether the priority is human readability, local accuracy, or compatibility with a specific model family.

| Method | Best For | Strength | Limitation |
|---|---|---|---|
| Direct feature overlap | Content-based filtering | Highly interpretable | Misses hidden non-linear effects |
| SHAP | Feature attribution | Strong local explanations | Extra computation cost |
| Neighborhood explanation | Collaborative filtering | Easy to verbalize | Can inherit neighborhood bias |
| LIME | Neural models | Model-agnostic | Can vary across runs |

*Table 28. Comparison of explainability methods used in the notebook.*

| Model | Sampled RMSE | Precision@10 | Recall@10 | Evaluated Users |
|---|---|---|---|---|
| Surprise SVD | 0.8115 | 0.3467 | 0.1247 | 15 |
| Item-CF (k=40) | 0.8178 | 0.3933 | 0.1564 | 15 |
| Item-CF (k=20) | 0.8252 | 0.3867 | 0.1479 | 15 |
| Item-CF (k=10) | 0.8279 | 0.3733 | 0.1149 | 15 |
| User-CF (k=20) | 0.8327 | 0.3600 | 0.1403 | 15 |
| User-CF (k=40) | 0.8409 | 0.3667 | 0.1429 | 15 |
| User-CF (k=10) | 0.8455 | 0.3333 | 0.1231 | 15 |
| Manual SVD | 0.8837 | 0.4200 | 0.1703 | 15 |
| Hybrid Meta-Model | 0.8901 | 0.3133 | 0.1186 | 15 |
| User-Profile Content Recommender | 0.9428 | 0.0000 | 0.0000 | 15 |
| Neural Content Model | 1.0621 | 0.2200 | 0.0512 | 15 |

*Table 29. Final model comparison sorted by sampled RMSE.*

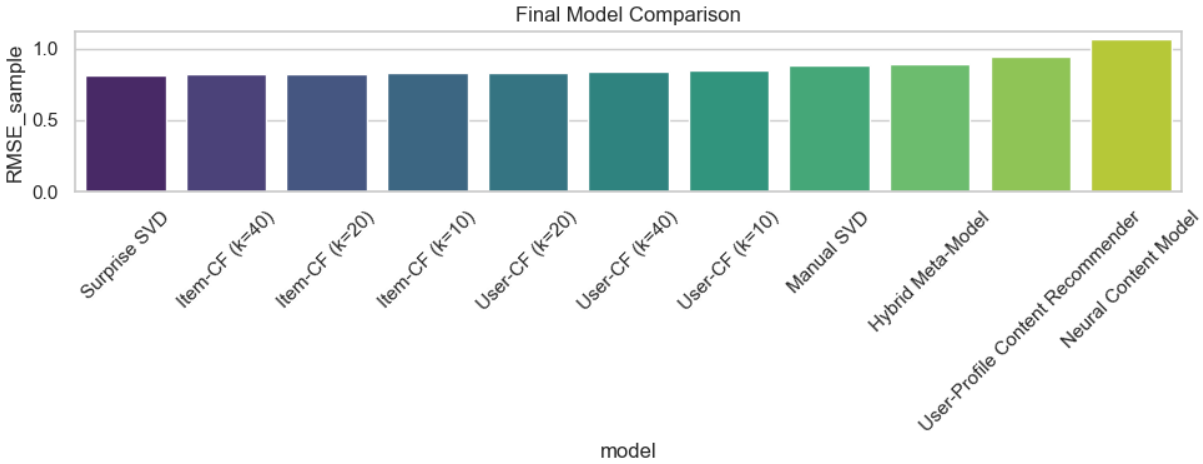


*Figure 9. Final model comparison across all recommenders in the notebook.*

**Task 13 result analysis**

Three findings stand out from the final comparison. First, Surprise SVD achieved the best sampled RMSE (0.8115), making it the best choice when rating-prediction accuracy is the priority. Second, Manual SVD achieved the best Precision@10 (0.4200) and Recall@10 (0.1703), showing that the best rating regressor is not always the best ranker. Third, content-only recommendation was the easiest to explain but the weakest on held-out ranking, whereas collaborative and factorization models offered stronger recommendation quality at the cost of lower inherent interpretability.

# Conclusion

The executed notebook successfully covers all thirteen tasks in the assignment using the MovieLens dataset. A clear progression emerges across the experiments: simple content-based methods are highly interpretable but limited by coarse metadata; collaborative filtering significantly improves recommendation quality; matrix factorization provides the strongest overall performance; and explainability tools are essential for making advanced recommenders auditable.

For this submitted run, Surprise SVD is the strongest choice for rating prediction, Manual SVD is the strongest choice for top-K ranking, and Item-CF (k = 40) is the strongest neighborhood baseline. If this system were extended further, the most promising improvements would be richer movie metadata, longer neural training, more hybrid training data, and production-oriented explainability layered on top of the strongest collaborative models.

# Appendix

Notebook execution environment and package versions captured in the submitted run

## Execution environment

The table below reproduces the package versions displayed at the start of the executed notebook. It is included for reproducibility and to document the software context of the submitted run.

| Package | Version |
|---|---|
| python | 3.12.4 |
| numpy | 1.26.4 |
| pandas | 2.2.2 |
| torch | 2.2.2+cpu |
| shap | 0.46.0 |

*Table 30. Package versions reported by the executed notebook.*